**1)Check if number a is present in number b**

**Example Input:**

a = 45 b = 1234567

**Output:**

Yes

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    int temp = b, rem, rev = 0;
    char str_a[20], str_b[20];
    sprintf(str_a, "%d", a);
    sprintf(str_b, "%d", b);
```

```
 if(strstr(str_b, str_a))
        printf("Yes\n");
    else
        printf("No\n");

    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class CheckNumberInNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();

        String strA = Integer.toString(a);
        String strB = Integer.toString(b);
```

```
 if(strB.contains(strA))
            System.out.println("Yes");
        else
            System.out.println("No");
    }
}
```

**2)Prime factor – Sort the array based on the smallest prime factor of each element**

**Example Input:**

Array: [15, 21, 10, 33]

**Explanation:**

•15 → min prime factor = 3

•21 → 3

•10 → 2

•33 → 3

**Sorted by min prime factor:**

[10, 15, 21, 33]

# ANSWER IN C

```
#include <stdio.h>
int minPrimeFactor(int n) {
    for(int i = 2; i <= n; i++)
        if(n % i == 0)
            return i;
    return n;
}
int main() {
    int n, arr[100];
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
for(int i = 0; i < n - 1; i++) {
        for(int j = i + 1; j < n; j++) {
            if(minPrimeFactor(arr[i]) > minPrimeFactor(arr[j])) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.*;

public class PrimeFactorSort {
    public static int minPrimeFactor(int n) {
        for(int i = 2; i <= n; i++)
            if(n % i == 0)
                return i;
        return n;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```java
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Integer[] arr = new Integer[n];
        for(int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        Arrays.sort(arr, Comparator.comparingInt(a -> minPrimeFactor(a)));

        for(int i : arr)
            System.out.print(i + " ");
    }
}
```

**3)Add a digit to all the digits of a number**

**Example:** Add digit 4 to each digit of 2875

**Input:**

digit = 4 number = 2875

**Output:**

612119

(2+4=6, 8+4=12, 7+4=11, 5+4=9 → concatenate: 6 12 11 9)

# ANSWER IN C

```c
#include <stdio.h>
int main() {
    int num, d;
    printf("Enter number and digit to add: ");
    scanf("%d %d", &num, &d);
    int rev = 0;
    while(num > 0) {
        rev = rev * 10 + num % 10;
        num /= 10;
    }
```

```
 printf("Output: ");
   while(rev > 0) {
       int x = rev % 10;
       printf("%d", x + d);
       rev /= 10;
   }
   return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class AddDigit {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = sc.nextInt();
        System.out.print("Enter digit to add: ");
        int d = sc.nextInt();

        StringBuilder sb = new StringBuilder();
        String s = Integer.toString(num);
```

```
 for(char c : s.toCharArray()) {
            int sum = (c - '0') + d;
            sb.append(sum);
        }

        System.out.println("Output: " + sb.toString());
    }
}
```

**4)Digit Removal Iteratively**

**Description:** Given a list of numbers and a list of digits, in each iteration:

- If a digit exists in any number, remove it.
- Continue for next digit until list is empty.

**Example Input:**

Numbers = [123, 245, 345, 456] Digits = [2, 4]

**After removing 2:**

- [13, 45, 345, 456]

**After removing 4:**

- [13, 5, 35, 56]

**Output:**

[13, 5, 35, 56]

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, d;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter numbers: ");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Enter number of digits to remove: ");
    scanf("%d", &d);
    int digits[d];
```

```c
printf("Enter digits to remove: ");
  for(int i = 0; i < d; i++)
      scanf("%d", &digits[i]);
  for(int k = 0; k < d; k++) {
      int digit = digits[k];
      for(int i = 0; i < n; i++) {
          int num = arr[i], res = 0, mul = 1;
          while(num > 0) {
              int r = num % 10;
              if(r != digit) {
                  res = r * mul + res;
                  mul *= 10;
              }
              num /= 10;
          }
          arr[i] = res;
      }
  }
```

```
  printf("Output: ");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class DigitRemovalIterative {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.print("Enter numbers: ");
        for(int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
```

```java
System.out.print("Enter number of digits to remove: ");
int d = sc.nextInt();
int[] digits = new int[d];

System.out.print("Enter digits to remove: ");
for(int i = 0; i < d; i++)
    digits[i] = sc.nextInt();

for(int digit : digits) {
    for(int i = 0; i < n; i++) {
        String s = Integer.toString(arr[i]).replaceAll(Integer.toString(digit),
"");
```

```
 arr[i] = s.isEmpty() ? 0 : Integer.parseInt(s);
            }
        }

        System.out.print("Output: ");
        for(int x : arr)
            System.out.print(x + " ");
    }
}
```

5)Form the largest possible number using the array of numbers

Example

Input:

Array = [3, 30, 34, 5, 9]

Output:

9534330

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int compare(const void *a, const void *b) {
    char ab[20], ba[20];
    sprintf(ab, "%d%d", *(int*)a, *(int*)b);
    sprintf(ba, "%d%d", *(int*)b, *(int*)a);
    return strcmp(ba, ab);
}
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
```

```c
printf("Enter numbers: ");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    qsort(arr, n, sizeof(int), compare);

    printf("Largest number: ");
    for(int i = 0; i < n; i++)
        printf("%d", arr[i]);

    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.*;
public class LargestNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        String[] arr = new String[n];

        System.out.print("Enter numbers: ");
        for(int i = 0; i < n; i++)
            arr[i] = sc.next();
```

```java
Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));

        StringBuilder sb = new StringBuilder();
        for(String s : arr)
            sb.append(s);

        System.out.println("Largest number: " + sb.toString());
    }
}
```

6)Given an amount, find the minimum number of notes of different denominations that sum up to the given amount. Starting from the highest denomination note, try to accommodate as many notes as possible for a given amount. We may assume that we have infinite supply of notes of values {2000, 500, 200, 100, 50, 20, 10, 5, 1}

**Examples:**

**Input** : 800
**Output** : Currency Count
500 : 1
200 : 1
100 : 1

**Input** : 2456
**Output** : Currency Count
2000 : 1
200 : 2
50 : 1
5 : 1
1 : 1

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int amount;
    int denominations[] = {2000, 500, 200, 100, 50, 20, 10, 5, 1};
    int n = sizeof(denominations)/sizeof(denominations[0]);
    printf("Enter the amount: ");
    scanf("%d", &amount);
    printf("Currency  Count\n");
```

```
for(int i = 0; i < n; i++) {
    if(amount >= denominations[i]) {
        int count = amount / denominations[i];
        amount = amount % denominations[i];
        printf("%d : %d\n", denominations[i], count);
    }
  }
  return 0;
}
```

# ANSWER IN JAVA

```java
import java.util.Scanner;

public class MinimumNotes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] denominations = {2000, 500, 200, 100, 50, 20, 10, 5, 1};
        System.out.print("Enter the amount: ");
        int amount = sc.nextInt();

        System.out.println("Currency  Count");
```

```
  for (int i = 0; i < denominations.length; i++) {
        if (amount >= denominations[i]) {
            int count = amount / denominations[i];
            amount = amount % denominations[i];
            System.out.println(denominations[i] + " : " + count);
        }
      }
      sc.close();
    }
}
```

**7)Problem:**

Given a number, find the sum of its digits.

**Example:**

Input: 1234

Output: 10

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int num, sum = 0, digit;
    printf("Enter a number: ");
    scanf("%d", &num);
    while (num != 0) {
        digit = num % 10;
        sum += digit;
        num /= 10;
    }
    printf("Sum of digits: %d\n", sum);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class SumOfDigits {
    public static void main(String[] args) {
        int num, sum = 0, digit;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        num = sc.nextInt();
        while (num != 0) {
            digit = num % 10;
            sum += digit;
            num /= 10;   }
        System.out.println("Sum of digits: " + sum);
    }
}
```

## 8) **Palindrome Check**

**Problem:**

Check whether a given string or number is a palindrome or not.

**Example:**

Input: madam

Output: Palindrome

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len, flag = 1;
    scanf("%s", str);
    len = strlen(str);
    for(i = 0; i < len / 2; i++) {
        if(str[i] != str[len - i - 1]) {
            flag = 0;
            break;
        }
    }
```

```
 if(flag)
        printf("Palindrome\n");
    else
        printf("Not Palindrome\n");

    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class PalindromeCheck {
    public static void main(String[] args) {
        String str;
        int len, flag = 1;
        Scanner sc = new Scanner(System.in);
        str = sc.next();
        len = str.length();
        for(int i = 0; i < len / 2; i++) {
            if(str.charAt(i) != str.charAt(len - i - 1)) {
                flag = 0;
                break;
            }
        }
```

```
 if(flag == 1)
            System.out.println("Palindrome");
        else
            System.out.println("Not Palindrome");
    }
}
```

# 9)Count Occurrences of a Character

**Problem:**

Input a string and a character, count its occurrences in the string.

**Example:**

Input: banana, a

Output: 3

# ANSWER IN C

```c
#include <stdio.h>
int main() {
    char str[100], ch;
    int i, count = 0;
    scanf("%s", str);
    scanf(" %c", &ch);
    for(i = 0; str[i] != '\0'; i++) {
        if(str[i] == ch)
            count++;
    }
    printf("%d\n", count);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class CountCharacterOccurrence {
    public static void main(String[] args) {
        String str;
        char ch;
        int count = 0;
        Scanner sc = new Scanner(System.in);
        str = sc.next();
        ch = sc.next().charAt(0);
        for(int i = 0; i < str.length(); i++) {
            if(str.charAt(i) == ch)
                count++;
        }
        System.out.println(count);
    }
}
```

## 10) **Fibonacci Series**

**Problem:**

Print Fibonacci series up to n terms.

**Example:**

Input: 5

Output: 0 1 1 2 3

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i, a = 0, b = 1, c;
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }
    return 0;
}
```

# ANSWER IN JAVA

```java
import java.util.Scanner;
public class Fibonacci {
    public static void main(String[] args) {
        int n, a = 0, b = 1, c;
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        for(int i = 0; i < n; i++) {
            System.out.print(a + " ");
            c = a + b;
            a = b;
            b = c;
        }
    }
}
```

11) Swap Two Numbers Without Using Third Variable

Problem:

Input: a=5, b=10

Output: a=10, b=5

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    a = a + b;
    b = a - b;
    a = a - b;
    printf("%d %d", a, b);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class Swap {
    public static void main(String[] args) {
        int a, b;
        Scanner sc = new Scanner(System.in);
        a = sc.nextInt();
        b = sc.nextInt();
        a = a + b;
        b = a - b;
        a = a - b;
        System.out.println(a + " " + b);
    }
}
```

## 12) **Remove Duplicate Characters**

**Problem:**

Remove duplicate characters from a string.

**Example:**

Input: programming

Output: progamin

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, j, k;
    scanf("%s", str);
```

```
for(i = 0; str[i] != '\0'; i++) {
        for(j = i + 1; str[j] != '\0'; ) {
            if(str[i] == str[j]) {
                for(k = j; str[k] != '\0'; k++)
                    str[k] = str[k + 1];
            } else {
                j++;
            }
        }
    }
    printf("%s", str);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class RemoveDuplicates {
    public static void main(String[] args) {
        String str;
        String result = "";
        Scanner sc = new Scanner(System.in);
        str = sc.next();
        for(int i = 0; i < str.length(); i++) {
            if(!result.contains(str.charAt(i) + ""))
                result += str.charAt(i);
        }
        System.out.println(result);
    }
}
```

## 13) **Check if two strings are anagrams**

**Question:**

Write a program to check whether two strings are anagrams of each other or not.

**Example:**

Input: "listen", "silent"

Output: Anagrams

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int cmp(const void *a, const void *b) {
    return (*(char*)a - *(char*)b);
}
int main() {
    char str1[100], str2[100];
    scanf("%s %s", str1, str2);
    if(strlen(str1) != strlen(str2)) {
        printf("Not Anagrams");
        return 0;
    }
```

```
 qsort(str1, strlen(str1), sizeof(char), cmp);
    qsort(str2, strlen(str2), sizeof(char), cmp);
    if(strcmp(str1, str2) == 0)
        printf("Anagrams");
    else
        printf("Not Anagrams");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
import java.util.Arrays;
public class Anagram {
    public static void main(String[] args) {
        String str1, str2;
        Scanner sc = new Scanner(System.in);
        str1 = sc.next();
        str2 = sc.next();
        if(str1.length() != str2.length()) {
            System.out.println("Not Anagrams");
            return;
        }
```

```
 char[] a = str1.toCharArray();
        char[] b = str2.toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        if(Arrays.equals(a, b))
            System.out.println("Anagrams");
        else
            System.out.println("Not Anagrams");
    }
}
```

14) Write a program to check if a given number is an Armstrong number.

**Example:**

Input: 153

Output: Armstrong Number

# ANSWER IN C

```
#include <stdio.h>
#include <math.h>
int main() {
    int num, temp, rem, sum = 0, n = 0;
    scanf("%d", &num);
    temp = num;
    while(temp != 0) {
        temp /= 10;
        n++;
    }
    temp = num;
```

```
while(temp != 0) {
        rem = temp % 10;
        sum += pow(rem, n);
        temp /= 10;
    }
    if(sum == num)
        printf("Armstrong Number");
    else
        printf("Not Armstrong Number");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class Armstrong {
    public static void main(String[] args) {
        int num, temp, rem, sum = 0, n = 0;
        Scanner sc = new Scanner(System.in);
        num = sc.nextInt();
        temp = num;
        while(temp != 0) {
            temp /= 10;
            n++;
        }
        temp = num;
```

```
 while(temp != 0) {
      rem = temp % 10;
      sum += Math.pow(rem, n);
      temp /= 10;
    }
    if(sum == num)
      System.out.println("Armstrong Number");
    else
      System.out.println("Not Armstrong Number");
  }
}
```

## 15) **Convert decimal to binary**

**Question:**

Write a program to convert a decimal number to its binary equivalent.

**Example:**

Input: 10

Output: 1010

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int num, bin[32], i = 0, j;
    scanf("%d", &num);
    if(num == 0) {
        printf("0");
        return 0;
    }
```

```
while(num != 0) {
        bin[i++] = num % 2;
        num /= 2;
    }
    for(j = i - 1; j >= 0; j--)
        printf("%d", bin[j]);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class DecimalToBinary {
    public static void main(String[] args) {
        int num;
        Scanner sc = new Scanner(System.in);
        num = sc.nextInt();
        if(num == 0) {
            System.out.println("0");
            return;
        }
```

```
String bin = "";
    while(num != 0) {
        bin = (num % 2) + bin;
        num /= 2;
    }
    System.out.println(bin);
  }
}
```

## 17) **Parentheses Balance Check**

**Question:**

Write a program to check whether the given string containing only ()[]{} is balanced or not.

**Example:**

Input: {[()]} → Balanced

Input: {[(])} → Not Balanced

# ANSWER IN C

```
#include <stdio.h>
#define SIZE 100
char stack[SIZE];
int top = -1;
void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peek() { return stack[top]; }
int isEmpty() { return top == -1; }
int main() {
    char str[SIZE];
    int i, balanced = 1;
    scanf("%s", str);
    for(i = 0; str[i] != '\0'; i++) {
        if(str[i] == '(' || str[i] == '{' || str[i] == '[')
            push(str[i]);
```

```
else {
      if(isEmpty()) {
        balanced = 0;
        break;
      }
      char c = pop();
      if((str[i] == ')' && c != '(') ||
         (str[i] == '}' && c != '{') ||
         (str[i] == ']' && c != '[')) {
        balanced = 0;
        break;
      }
    }
  }
```

```
 if(!isEmpty()) balanced = 0;
    if(balanced)
        printf("Balanced");
    else
        printf("Not Balanced");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
import java.util.Stack;
public class ParenthesesBalance {
    public static void main(String[] args) {
        String str;
        Scanner sc = new Scanner(System.in);
        str = sc.next();
        Stack<Character> stack = new Stack<>();
        boolean balanced = true;
        for(int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            if(c == '(' || c == '{' || c == '[')
                stack.push(c);
```

```
else {
           if(stack.isEmpty()) {
              balanced = false;
              break;
           }
           char top = stack.pop();
           if((c == ')' && top != '(') ||
             (c == '}' && top != '{') ||
             (c == ']' && top != '[')) {
              balanced = false;
              break;
           }
        }
     }
```

```
  if(!stack.isEmpty()) balanced = false;
      if(balanced)
          System.out.println("Balanced");
      else
          System.out.println("Not Balanced");
   }
}
```

## 18) **Encode the String**

**Question:**

Write a program to encode a given string by replacing continuous occurrence of character with character followed by count.

**Example:**

Input: "aaabbcddd"

Output: "a3b2c1d3"

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, count;
    scanf("%s", str);
    for(i = 0; i < strlen(str); ) {
        char c = str[i];
        count = 0;
```

```
while(str[i] == c) {
            count++;
            i++;
        }
        printf("%c%d", c, count);
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class EncodeString {
    public static void main(String[] args) {
        String str;
        Scanner sc = new Scanner(System.in);
        str = sc.next();
        int i = 0;
        while(i < str.length()) {
            char c = str.charAt(i);
            int count = 0;
```

```
while(i < str.length() && str.charAt(i) == c) {
            count++;
            i++;
        }
        System.out.print(c + "" + count);
    }
  }
}
```

# 19) **Implement atoi() function**

**Question:**

Write your own version of atoi() function in C or Java.

# ANSWER IN C

```
#include <stdio.h>
int myAtoi(char *str) {
    int res = 0, i = 0, sign = 1;
    if(str[0] == '-') {
        sign = -1;
        i++;
    }for(; str[i] != '\0'; i++)
        res = res * 10 + (str[i] - '0');
    return sign * res;}
int main() {
    char str[100];
    scanf("%s", str);
    printf("%d", myAtoi(str));
    return 0;}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class MyAtoi {
    public static int myAtoi(String str) {
        int res = 0, i = 0, sign = 1;
        if(str.charAt(0) == '-') {
            sign = -1;
            i++;
        }
```

```
for(; i < str.length(); i++)
        res = res * 10 + (str.charAt(i) - '0');
    return sign * res;
  }
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String str = sc.next();
    System.out.println(myAtoi(str));
  }
}
```

# 20)Merge Two Sorted Arrays Without Extra Space

**Question:**

Given two sorted arrays, merge them without using extra space (modify in-place).

**Example:**

Input: {1,3,5}, {2,4,6}

Output: {1,2,3,4,5,6}

# ANSWER IN C

```
#include <stdio.h>
#include <math.h>
int nextGap(int gap) {
    if (gap <= 1) return 0;
    return (gap / 2) + (gap % 2);
}
int main() {
    int n, m, i, j, gap;
    int a[100], b[100];
    scanf("%d", &n);
    for(i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &m);
    for(i = 0; i < m; i++) scanf("%d", &b[i]);
```

```
gap = nextGap(n + m);
  while(gap > 0) {
     for(i = 0; i + gap < n; i++) {
        if(a[i] > a[i + gap]) {
           int temp = a[i];
           a[i] = a[i + gap];
           a[i + gap] = temp;
        }
     }
     for(j = gap > n ? gap - n : 0; i < n && j < m; i++, j++) {
        if(a[i] > b[j]) {
           int temp = a[i];
           a[i] = b[j];
           b[j] = temp;
        }
     }
```

```
 if(j < m) {
        for(j = 0; j + gap < m; j++) {
          if(b[j] > b[j + gap]) {
            int temp = b[j];
            b[j] = b[j + gap];
            b[j + gap] = temp;
          }
        }
      }
      gap = nextGap(gap);
    }

    for(i = 0; i < n; i++) printf("%d ", a[i]);
    for(i = 0; i < m; i++) printf("%d ", b[i]);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class MergeSortedArrays {
    public static int nextGap(int gap) {
        if(gap <= 1) return 0;
        return (gap / 2) + (gap % 2);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), m = sc.nextInt();
        int[] a = new int[n], b = new int[m];
```

```
for(int i = 0; i < n; i++) a[i] = sc.nextInt();
    for(int i = 0; i < m; i++) b[i] = sc.nextInt();

    int gap = nextGap(n + m);
    while(gap > 0) {
        int i, j;
        for(i = 0; i + gap < n; i++) {
            if(a[i] > a[i + gap]) {
                int temp = a[i];
                a[i] = a[i + gap];
                a[i + gap] = temp;
            }
        }
```

```
for(j = gap > n ? gap - n : 0; i < n && j < m; i++, j++) {
            if(a[i] > b[j]) {
                int temp = a[i];
                a[i] = b[j];
                b[j] = temp;
            }
        }
        if(j < m) {
            for(j = 0; j + gap < m; j++) {
                if(b[j] > b[j + gap]) {
                    int temp = b[j];
                    b[j] = b[j + gap];
                    b[j + gap] = temp;
                }
            }
        }
```

```
  gap = nextGap(gap);
        }

        for(int x : a) System.out.print(x + " ");
        for(int x : b) System.out.print(x + " ");
    }
}
```

## 21) **Longest Common Prefix**

**Question:**

Write a program to find the longest common prefix among an array of strings.

**Example:**

Input: {“flower”,”flow”,”flight”}

Output: "fl"

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    int n, i, j;
    char str[100][100], prefix[100];
    scanf("%d", &n);
    for(i = 0; i < n; i++)
        scanf("%s", str[i]); strcpy(prefix, str[0]);
    for(i = 1; i < n; i++) {
        j = 0;
```

```
  while(prefix[j] && str[i][j] && prefix[j] == str[i][j])
          j++;
      prefix[j] = '\0';
    }
    printf("%s", prefix);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class LongestCommonPrefix {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        String[] strs = new String[n];
        for(int i = 0; i < n; i++)
            strs[i] = sc.next();

        String prefix = strs[0];
```

```
  for(int i = 1; i < n; i++) {
            int j = 0;
            while(j < prefix.length() && j < strs[i].length() && 
prefix.charAt(j) == strs[i].charAt(j))
                j++;
            prefix = prefix.substring(0, j);
        }
        System.out.println(prefix);
    }
}
```

## 22) **Spiral Order Matrix Traversal**

**Question:**

Print elements of a matrix in spiral order.

Sample Input:

Matrix:

1 2 3

4 5 6

7 8 9

Output (Spiral Order)

1 2 3 6 9 8 7 4 5

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int r, c, i, rowStart, rowEnd, colStart, colEnd;
    int a[100][100];
    scanf("%d %d", &r, &c);
    for(i = 0; i < r; i++)
        for(int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    rowStart = 0;
    rowEnd = r - 1;
    colStart = 0;
```

```
colEnd = c - 1;

 while(rowStart <= rowEnd && colStart <= colEnd) {
    for(i = colStart; i <= colEnd; i++)
        printf("%d ", a[rowStart][i]);
    rowStart++;
    for(i = rowStart; i <= rowEnd; i++)
        printf("%d ", a[i][colEnd]);
    colEnd--;
    if(rowStart <= rowEnd) {
        for(i = colEnd; i >= colStart; i--)
            printf("%d ", a[rowEnd][i]);
        rowEnd--;
    }
```

```
    if(colStart <= colEnd) {
            for(i = rowEnd; i >= rowStart; i--)
                printf("%d ", a[i][colStart]);
            colStart++;
        }
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class SpiralOrder {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int r = sc.nextInt(), c = sc.nextInt();
        int[][] a = new int[r][c];
        for(int i = 0; i < r; i++)
            for(int j = 0; j < c; j++)
                a[i][j] = sc.nextInt();

        int rowStart = 0, rowEnd = r - 1, colStart = 0, colEnd = c - 1;
```

```
while(rowStart <= rowEnd && colStart <= colEnd) {
        for(int i = colStart; i <= colEnd; i++)
            System.out.print(a[rowStart][i] + " ");
        rowStart++;
        for(int i = rowStart; i <= rowEnd; i++)
            System.out.print(a[i][colEnd] + " ");
        colEnd--;
        if(rowStart <= rowEnd) {
            for(int i = colEnd; i >= colStart; i--)
                System.out.print(a[rowEnd][i] + " ");
            rowEnd--;
        }
```

```
    if(colStart <= colEnd) {
            for(int i = rowEnd; i >= rowStart; i--)
                System.out.print(a[i][colStart] + " ");
            colStart++;
        }
      }
    }
  }
```

23)**Replace each element with product of all other elements**

**Question:**

Given an array, replace every element with the product of all other elements without using division.

**Example:**

Input: {1,2,3,4,5}

Output: {120,60,40,30,24}

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i;
    int a[100], left[100], right[100], prod[100];
    scanf("%d", &n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    left[0] = 1;
    for(i = 1; i < n; i++)
        left[i] = left[i - 1] * a[i - 1];
    right[n - 1] = 1;
```

```
  for(i = n - 2; i >= 0; i--)
      right[i] = right[i + 1] * a[i + 1];

   for(i = 0; i < n; i++)
      prod[i] = left[i] * right[i];

   for(i = 0; i < n; i++)
      printf("%d ", prod[i]);
   return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class ProductArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n], left = new int[n], right = new int[n], prod = new int[n];
        for(int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        left[0] = 1;
        for(int i = 1; i < n; i++)
            left[i] = left[i - 1] * a[i - 1];
        right[n - 1] = 1;
```

```
  for(int i = n - 2; i >= 0; i--)
          right[i] = right[i + 1] * a[i + 1];

      for(int i = 0; i < n; i++)
          prod[i] = left[i] * right[i];

      for(int x : prod)
          System.out.print(x + " ");
    }
}
```

24) **Rearrange array in max-min form**

**Question:**

Rearrange a sorted array in max/min form alternately.

✅ **Example:**

Input: {1,2,3,4,5,6,7}

Output: {7,1,6,2,5,3,4}

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i;
    int a[100], res[100];
    scanf("%d", &n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    int start = 0, end = n - 1, k = 0;
    while(start <= end) {
        if(k % 2 == 0)
```

```
    res[k++] = a[end--];
        else
            res[k++] = a[start++];
    }

    for(i = 0; i < n; i++)
        printf("%d ", res[i]);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class MaxMinRearrange {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n], res = new int[n];
        for(int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int start = 0, end = n - 1, k = 0;
```

```
  while(start <= end) {
          if(k % 2 == 0)
              res[k++] = a[end--];
          else
              res[k++] = a[start++];
      }

      for(int x : res)
          System.out.print(x + " ");
    }
}
```

25) **Move all zeros to end of array**

**Question:**

Write a program to move all zeroes in an array to the end without changing order of non-zero elements.

✅ **Example:**

Input: {1,0,2,0,3,0,4,5}

Output: {1,2,3,4,5,0,0,0}

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i, j = 0;
    int a[100];
    scanf("%d", &n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for(i = 0; i < n; i++) {
        if(a[i] != 0)
            a[j++] = a[i];
    }
```

```
    while(j < n)
        a[j++] = 0;

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class MoveZerosEnd {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for(int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        int j = 0;
        for(int i = 0; i < n; i++) {
            if(a[i] != 0)
                a[j++] = a[i];
        }
```

```
    while(j < n)
        a[j++] = 0;

    for(int x : a)
        System.out.print(x + " ");
  }
}
```

26) **Reverse words in a string**

**Question:**

Reverse words of a string in-place without using split.

✅ **Example:**

Input: "I am Zoho student"

Output: "student Zoho am I"

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
void reverse(char *s, int start, int end) {
    while(start < end) {
        char temp = s[start];
        s[start++] = s[end];
        s[end--] = temp;
    }
}
int main() {
    char str[200];
    int i = 0, j = 0, len;
```

```
 fgets(str, sizeof(str), stdin);
len = strlen(str);
if(str[len-1] == '\n') str[len-1] = '\0';
len = strlen(str);

reverse(str, 0, len-1);

while(i < len) {
    if(str[i] != ' ') {
        j = i;
        while(j < len && str[j] != ' ')
            j++;
        reverse(str, i, j-1);
```

```
      i = j;
    } else i++;
  }

  printf("%s", str);
  return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class ReverseWords {
    public static void reverse(char[] s, int start, int end) {
        while(start < end) {
            char temp = s[start];
            s[start++] = s[end];
            s[end--] = temp;
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
```

```
char[] s = str.toCharArray();
    int n = s.length;
    reverse(s, 0, n - 1);
    int i = 0;
    while(i < n) {
        if(s[i] != ' ') {
            int j = i;
            while(j < n && s[j] != ' ')
                j++;
            reverse(s, i, j - 1);
```

```
    i = j;
        } else i++;
      }
      System.out.println(new String(s));
    }
}
```

27) **Find missing number in array 1 to n**

**Question:**

Find missing number in array containing numbers from 1 to n.

**Example:**

Input: {1,2,4,5,6}

Output: 3

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i, sum = 0, total;
    int a[100];
    scanf("%d", &n);
    for(i = 0; i < n - 1; i++) {
        scanf("%d", &a[i]);
        sum += a[i];}
    total = n * (n + 1) / 2;
    printf("%d", total - sum);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class MissingNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), sum = 0;
        int[] a = new int[n - 1];
        for(int i = 0; i < n - 1; i++) {
            a[i] = sc.nextInt();
            sum += a[i]; }
        int total = n * (n + 1) / 2;
        System.out.println(total - sum);
    }
}
```

## 28) **First non-repeating character in string**

**Question:**

Find first non-repeating character in a string.

**Example:**

Input: "aabbcddee"

Output: "c"

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int freq[256] = {0}, i;
    scanf("%s", str);
    for(i = 0; str[i]; i++)
        freq[(int)str[i]]++;
```

```
for(i = 0; str[i]; i++) {
        if(freq[(int)str[i]] == 1) {
            printf("%c", str[i]);
            break;
        }
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class FirstNonRepeat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int[] freq = new int[256];
        for(int i = 0; i < str.length(); i++)
            freq[str.charAt(i)]++;
```

```
for(int i = 0; i < str.length(); i++) {
        if(freq[str.charAt(i)] == 1) {
            System.out.println(str.charAt(i));
            break;
        }
      }
   }
}
```

## 29) **Find second largest element without sorting**

**Problem:** Find second largest in an array without using sort.

Example:

Input: {10,20,4,45,99}

Output: 45

# ANSWER IN C

```
#include <stdio.h>

int main() {
    int n, i, first = -1e9, second = -1e9, a[100];
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        if(a[i] > first) {
            second = first;
            first = a[i];
```

```
  } else if(a[i] > second && a[i] != first)
          second = a[i];
   }
   printf("%d", second);
   return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class SecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), first = Integer.MIN_VALUE, second = 
Integer.MIN_VALUE;
        int[] a = new int[n];
        for(int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
            if(a[i] > first) {
                second = first;
                first = a[i];
```

```
 } else if(a[i] > second && a[i] != first)
            second = a[i];
        }
        System.out.println(second);
    }
}
```

# 30)**Reverse a string**

**Problem:** Reverse a string without using inbuilt reverse.

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100], temp;
    int i, j;
    scanf("%s", str);
    i = 0;
    j = strlen(str) - 1;
```

```
while(i < j) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
        i++;
        j--;
    }
    printf("%s", str);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class ReverseString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        char[] s = str.toCharArray();
        int i = 0, j = s.length - 1;
```

```
while(i < j) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
    System.out.println(new String(s));
  }
}
```

## 31) **Sum of digits until single digit**

**Problem:** Keep summing digits until single digit is obtained.

Example:

Input: 9875

Process: 9+8+7+5=29 → 2+9=11 → 1+1=2

Output: 2

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, sum;
    scanf("%d", &n);
    while(n >= 10) {
        sum = 0;
        while(n > 0) {
            sum += n % 10;
            n /= 10; }
        n = sum;}
    printf("%d", n);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class SumUntilSingleDigit {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        while(n >= 10) {
            int sum = 0;
            while(n > 0) {
                sum += n % 10;
                n /= 10;}
            n = sum; }
        System.out.println(n);
    }}
```

# 32)Print diamond pattern

Problem: For input n=3:

```
  *
 * *
* * *
 * *
  *
```

# ANSWER IN C

```
#include <stdio.h>


int main() {
    int n, i, j;
    scanf("%d", &n);
    for(i = 1; i <= n; i++) {
        for(j = i; j < n; j++)
            printf(" ");
        for(j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
```

```
for(i = n - 1; i >= 1; i--) {
      for(j = n; j > i; j--)
         printf(" ");
      for(j = 1; j <= i; j++)
         printf("* ");
      printf("\n");
   }
   return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class DiamondPattern {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i = 1; i <= n; i++) {
            for(int j = i; j < n; j++)
                System.out.print(" ");
            for(int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
```

```
for(int i = n - 1; i >= 1; i--) {
        for(int j = n; j > i; j--)
            System.out.print(" ");
        for(int j = 1; j <= i; j++)
            System.out.print("* ");
        System.out.println();
      }
    }
}
```

## 33) **Prime number check**

**Problem:** Check if a given number is prime.

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i, flag = 1;
    scanf("%d", &n);
    if(n <= 1)
        flag = 0;
    for(i = 2; i * i <= n; i++) {
        if(n % i == 0) {
            flag = 0;
            break;
        }
    }
```

```
  if(flag)
        printf("Prime");
    else
        printf("Not Prime");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class PrimeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), flag = 1;
        if(n <= 1)
            flag = 0;
        for(int i = 2; i * i <= n; i++) {
            if(n % i == 0) {
                flag = 0;
                break;
            }
        }
```

```
 if(flag == 1)
          System.out.println("Prime");
      else
          System.out.println("Not Prime");
   }
}
```

## 34) **Count vowels and consonants**

**Problem:** Count number of vowels and consonants in a string.

# ANSWER IN C

```
#include <stdio.h>

int main() {
    char str[100];
    int i, v = 0, c = 0;
    scanf("%s", str);
    for(i = 0; str[i]; i++) {
        char ch = str[i];
        if((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
```

```
        if(ch == 'a'|| ch == 'e'|| ch == 'i'|| ch == 'o'|| ch == 'u'||
              ch == 'A'|| ch == 'E'|| ch == 'I'|| ch == 'O'|| ch == 'U')
              v++;
          else
              c++;
      }
   }
   printf("%d %d", v, c);
   return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class VowelConsonantCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int v = 0, c = 0;
        for(int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
```

```
  if(Character.isLetter(ch)) {
        if("aeiouAEIOU".indexOf(ch) != -1)
          v++;
        else
          c++;
      }
    }
    System.out.println(v + " " + c);
  }
}
```

35) **Pattern Printing**

**Problem:** Print the following pattern for n=4:

1

12

123

1234

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i, j;
    scanf("%d", &n);
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= i; j++)
            printf("%d", j);
        printf("\n");
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class PatternPrint {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= i; j++)
                System.out.print(j);
            System.out.println();
        }
    }
}
```

36) **Replace spaces with underscores**

**Problem:** Replace all spaces in string with underscores.

# ANSWER IN C

```
#include <stdio.h>
int main() {
    char str[200];
    fgets(str, sizeof(str), stdin);
    for(int i = 0; str[i]; i++) {
        if(str[i] == ' ')
            str[i] = '_';
    }
    printf("%s", str);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class ReplaceSpace {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        str = str.replace(' ', '_');
        System.out.println(str);
    }
}
```

37) **First non-repeating character**

**Problem:** Find first non-repeating character in string.

Example:

Input: "aabbcddee"

Output: "c"

# ANSWER IN C

```
#include <stdio.h>
int main() {
    char str[100];
    int freq[256] = {0}, i;
    scanf("%s", str);
    for(i = 0; str[i]; i++)
        freq[(int)str[i]]++;
```

```
for(i = 0; str[i]; i++) {
        if(freq[(int)str[i]] == 1) {
            printf("%c", str[i]);
            break;
        }
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class FirstNonRepeat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int[] freq = new int[256];
        for(int i = 0; i < str.length(); i++)
            freq[str.charAt(i)]++;
```

```
for(int i = 0; i < str.length(); i++) {
        if(freq[str.charAt(i)] == 1) {
            System.out.println(str.charAt(i));
            break;
        }
      }
   }
}
```

# 38)**GCD and LCM**

**Problem:** Find GCD and LCM of two numbers.

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int a, b, x, y, gcd, lcm;
    scanf("%d%d", &a, &b);
    x = a;
    y = b;
    while(b != 0) {
```

```
int temp = b;
        b = a % b;
        a = temp;
    }
gcd = a;
    lcm = (x * y) / gcd;
    printf("%d %d", gcd, lcm);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class GCDLCM {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt(), b = sc.nextInt();
        int x = a, y = b;
        while(b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
```

```
int gcd = a;
        int lcm = (x * y) / gcd;
        System.out.println(gcd + " " + lcm);
    }
}
```

39)**Matrix addition**

**Problem:** Add two matrices and print result.

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int r, c, i, j, a[10][10], b[10][10], sum[10][10];
    scanf("%d%d", &r, &c);
    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            scanf("%d", &b[i][j]);
```

```
for(i = 0; i < r; i++) {
        for(j = 0; j < c; j++)
            printf("%d ", a[i][j] + b[i][j]);
        printf("\n");
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class MatrixAddition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int r = sc.nextInt(), c = sc.nextInt();
        int[][] a = new int[r][c], b = new int[r][c];
        for(int i = 0; i < r; i++)
            for(int j = 0; j < c; j++)
                a[i][j] = sc.nextInt();
```

```
for(int i = 0; i < r; i++)
            for(int j = 0; j < c; j++)
                b[i][j] = sc.nextInt();
        for(int i = 0; i < r; i++) {
            for(int j = 0; j < c; j++)
                System.out.print((a[i][j] + b[i][j]) + " ");
            System.out.println();
        }
    }
}
```

# 40) **Subarray with given sum**

**Problem:** Find continuous subarray adding up to given sum (sliding window).

Example:

Input: {1,4,20,3,10,5}, sum=33

Output: 20,3,10

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, sum, a[100], s = 0, start = 0;
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &sum);
    for(int i = 0; i < n; i++) {
        s += a[i];
```

```
 while(s > sum)
            s -= a[start++];
 if(s == sum) {
            for(int j = start; j <= i; j++)
                printf("%d ", a[j]);
            break;
        }
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class SubarraySum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), s = 0, start = 0;
        int[] a = new int[n];
        for(int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        int sum = sc.nextInt();
        for(int i = 0; i < n; i++) {
            s += a[i];
```

```
while(s > sum)
        s -= a[start++];
      if(s == sum) {
        for(int j = start; j <= i; j++)
          System.out.print(a[j] + " ");
        break;
      }
    }
  }
}
```

41) **Count number of set bits**

**Problem:** Count number of 1s in binary representation.

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, count = 0;
    scanf("%d", &n);
    while(n) {
        count += n & 1;
        n >>= 1;
    }
    printf("%d", count);
    return 0;}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class CountSetBits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), count = 0;
        while(n != 0) {
            count += n & 1;
            n >>= 1;
        }
        System.out.println(count);
    }
}
```

# 42) **Factorial**

**Problem:** Find factorial of a number using loops or recursion.

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, i;
    long long fact = 1;
    scanf("%d", &n);
    for(i = 1; i <= n; i++)
        fact *= i;
    printf("%lld", fact);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class Factorial {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long fact = 1;
        for(int i = 1; i <= n; i++)
            fact *= i;
        System.out.println(fact);
    }
}
```

## 43) **Reverse a number**

**Problem:** Reverse the digits of a given number without converting to string.

Example:

Input: 1234

Output: 4321

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, rev = 0;
    scanf("%d", &n);
    while(n != 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    printf("%d", rev);
    return 0; }
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), rev = 0;
        while(n != 0) {
            rev = rev * 10 + n % 10;
            n /= 10;
        }
        System.out.println(rev);
    }
}
```

## 44) **Palindrome number**

**Problem:** Check if a number is a palindrome.

Example:

Input: 121

Output: Palindrome

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, rev = 0, temp, rem;
    scanf("%d", &n);
    temp = n;
    while(temp != 0) {
        rem = temp % 10;
        rev = rev * 10 + rem;
        temp /= 10;
    }
```

```c
 if(n == rev)
        printf("Palindrome");
    else
        printf("Not Palindrome");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class PalindromeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), temp = n, rev = 0;
        while(temp != 0) {
            rev = rev * 10 + temp % 10;
            temp /= 10;
        }
```

```
  if(n == rev)
          System.out.println("Palindrome");
      else
          System.out.println("Not Palindrome");
   }
}
```

## 45) **Check Even or Odd**

**Problem:**

Determine if a number is even or odd.

Example:

Input: 7

Output: Odd

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    if(n % 2 == 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class EvenOdd {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(n % 2 == 0)
            System.out.println("Even");
        else
            System.out.println("Odd");
    }
}
```

# 46) Print inverted pyramid

## Problem:

```
* * * *
* * *
* *
*
```

# ANSWER IN C

```c
#include <stdio.h>
int main() {
    int n, i, j;
    scanf("%d", &n);
    for(i = n; i >= 1; i--) {
        for(j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
    return 0;}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class InvertedPyramid {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i = n; i >= 1; i--) {
            for(int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

47) **Find duplicates in array without extra space**

Input: {4,3,2,7,8,2,3,1}

Output: 2 3

# ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, a[100];
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for(int i = 0; i < n; i++) {
        int index = abs(a[i]) - 1;
        if(a[index] < 0)
```

```
  printf("%d ", abs(a[i]));
        else
            a[index] = -a[index];
    }
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class FindDuplicates {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for(int i = 0; i < n; i++)
            a[i] = sc.nextInt();
```

```
for(int i = 0; i < n; i++) {
        int index = Math.abs(a[i]) - 1;
        if(a[index] < 0)
            System.out.print(Math.abs(a[i]) + " ");
        else
            a[index] = -a[index];
    }
  }
}
```

48) **Merge two sorted arrays without extra space**

Input: {1,3,5}, {2,4,6}

Output: {1,2,3,4,5,6}

# ANSWER IN C

```
#include <stdio.h>
int nextGap(int gap) {
    if(gap <= 1) return 0;
    return (gap / 2) + (gap % 2);
}
int main() {
    int n, m, i, j, gap, a[100], b[100];
    scanf("%d", &n);
```

```
for(i = 0; i < n; i++)
      scanf("%d", &a[i]);
  scanf("%d", &m);
  for(i = 0; i < m; i++)
      scanf("%d", &b[i]);
  gap = nextGap(n + m);
  while(gap > 0) {
      for(i = 0; i + gap < n; i++)
          if(a[i] > a[i + gap]) {
              int temp = a[i];
              a[i] = a[i + gap];
              a[i + gap] = temp;
          }
```

```
for(j = gap > n ? gap - n : 0; i < n && j < m; i++, j++)
        if(a[i] > b[j]) {
            int temp = a[i];
            a[i] = b[j];
            b[j] = temp;
        }
    for(j = 0; j + gap < m; j++)
        if(b[j] > b[j + gap]) {
            int temp = b[j];
            b[j] = b[j + gap];
            b[j + gap] = temp;
        }
    gap = nextGap(gap);
  }
```

```
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    for(i = 0; i < m; i++)
        printf("%d ", b[i]);
    return 0;
}
```

# ANSWER IN JAVA

```
public class MergeSortedArrays {
    static int nextGap(int gap) {
        if(gap <= 1) return 0;
        return (gap / 2) + (gap % 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
```

```
for(int i = 0; i < n; i++)
        a[i] = sc.nextInt();
    int m = sc.nextInt();
    int[] b = new int[m];
    for(int i = 0; i < m; i++)
        b[i] = sc.nextInt();
    int gap = nextGap(n + m);
    while(gap > 0) {
        int i, j;
        for(i = 0; i + gap < n; i++)
            if(a[i] > a[i + gap]) {
                int temp = a[i];
                a[i] = a[i + gap];
                a[i + gap] = temp;
            }
```

```
for(j = gap > n ? gap - n : 0; i < n && j < m; i++, j++)
            if(a[i] > b[j]) {
                int temp = a[i];
                a[i] = b[j];
                b[j] = temp;
            }
        for(j = 0; j + gap < m; j++)
            if(b[j] > b[j + gap]) {
                int temp = b[j];
                b[j] = b[j + gap];
                b[j + gap] = temp;
            }
        gap = nextGap(gap);
    }
```

```
for(i = 0; i < n; i++)
        System.out.print(a[i] + " ");
    for(i = 0; i < m; i++)
        System.out.print(b[i] + " ");
  }
}
```

49) **Longest substring without repeating characters**

Input: "abcabcbb"

Output: 3

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i, start = 0, maxLen = 0, index[256];
    for(i = 0; i < 256; i++)
        index[i] = -1;
    scanf("%s", s);
    for(i = 0; s[i]; i++) {
```

```
  if(index[s[i]] >= start)
            start = index[s[i]] + 1;
        index[s[i]] = i;
        if(i - start + 1 > maxLen)
            maxLen = i - start + 1;
    }
    printf("%d", maxLen);
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;

public class LongestUniqueSubstr {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        int[] index = new int[256];
        for(int i = 0; i < 256; i++)
            index[i] = -1;
        int maxLen = 0, start = 0;
```

```
for(int i = 0; i < s.length(); i++) {
        if(index[s.charAt(i)] >= start)
            start = index[s.charAt(i)] + 1;
        index[s.charAt(i)] = i;
        if(i - start + 1 > maxLen)
            maxLen = i - start + 1;
    }
    System.out.println(maxLen);
  }
}
```

## 50) **Check if two strings are rotations of each other**

Input: "ABCD","CDAB"

Output: True

# ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[100], s2[100], concat[200];
    scanf("%s%s", s1, s2);
    if(strlen(s1) != strlen(s2)) {
        printf("False");
        return 0;
    }
```

```
  strcpy(concat, s1);
    strcat(concat, s1);
    if(strstr(concat, s2))
        printf("True");
    else
        printf("False");
    return 0;
}
```

# ANSWER IN JAVA

```
import java.util.Scanner;
public class RotationCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = sc.next(), s2 = sc.next();
        if(s1.length() != s2.length())
            System.out.println("False");
         else if((s1 + s1).contains(s2))
            System.out.println("True");
        else
            System.out.println("False");
    } }
```